Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 5

La bandeja del revisor

DOCENTE:

Michael Montgomery Rosell

CURSO:

Desarrollo de Aplicaciones Empresariales

4-C24-A

I. Capacidades

a. **Capacidad terminal:**
Implementar en el administrador de Django la revisión humana de los cambios que propone un asistente de IA, con un verificador de reglas, permisos mínimos para el agente y registro de cada decisión.

Sub-componentes:
- Configura el administrador de Django para que una persona revise y decida sobre los cambios que propone un asistente de IA, y lo comprueba con pruebas automáticas.
- Separa con usuarios, grupos y permisos lo que puede hacer un agente de IA de lo que puede hacer una persona, y deja registro de cada decisión.
- Clasifica con evidencia los fallos de un código escrito por una IA y lo corrige de rojo a verde sin tocar las pruebas.

II. Seguridad

a. En este laboratorio está prohibida la manipulación del hardware y de las conexiones eléctricas o de red de los equipos, así como la ingestión de alimentos o bebidas en las estaciones de trabajo.
Ubicar mochilas y maletines en el lugar destinado para ello, de modo que no obstruyan el paso ni las salidas.
Al terminar la sesión, cerrar las sesiones abiertas en el equipo —incluidas las del repositorio y las de los servicios en línea que use la aplicación— y dejar la estación de trabajo y la silla limpias y ordenadas.

III. Fundamento teórico

a. Revisar, antes de iniciar el laboratorio, el material de la sesión de aula de la semana publicado en el campus virtual: la presentación del tema y el cuestionario de refuerzo. Cada laboratorio implementa lo que la sesión de aula presentó; sin ese repaso, el tiempo de laboratorio se va en entender el concepto en vez de en construirlo.
El proyecto del laboratorio es acumulativo: cada semana se parte del resultado de la anterior, no de un proyecto en blanco.

IV. Normas empleadas

a. Código Python según PEP 8 y la estructura de proyecto de Django. Los nombres de modelos, campos y funciones siguen en español, como en la guía del Lab 5 regular.
Las pruebas automáticas del repositorio son el criterio de aceptación: no se modifican.

V. Recursos

a. Cada estudiante trabaja en su equipo con Windows 10 o superior, Python 3.12 o
superior, Git y Visual Studio Code instalados, y con acceso a internet.

Cuentas necesarias: una cuenta de GitHub para el repositorio del laboratorio.

VI. Metodología para el desarrollo de la tarea

a. El laboratorio extra es individual y se desarrolla fuera de la sesión en vivo, después de ver la demostración del bloque de IA de la clase.
El proyecto se sube a un repositorio de GitHub propio. El informe registra, en este orden, la evidencia de cada parte: captura del resultado, código, explicación y la salida de `python manage.py duelo`, además de las tablas que pide el procedimiento.
El entregable se sube al campus virtual dentro del plazo de la semana, como segunda entrega de laboratorio de la semana 5.

VII. Procedimiento

1. Descargar el repositorio del laboratorio extra y, dentro de su carpeta, crear y activar el entorno virtual (python -m venv myvenv y myvenv\Scripts\activate), instalar Django con pip install django y preparar la base con python manage.py migrate, python manage.py preparar_tienda y python manage.py createsuperuser. La tienda trae las categorías y los productos del Lab 5 regular, y los usuarios asistente_ia y revisor (contraseña Tecsup2019).
2. Leer tienda/datos/sugerencias_ia.json: son las nueve respuestas que dio un asistente de IA cuando se le pidió revisar precios y stock de la tienda. Antes de escribir código, anotar en una tabla, para cada una, si la aplicarías en una tienda real y por qué. Esa tabla se compara al final con lo que decidió tu programa.
3. Ejecutar python manage.py duelo y copiar el resumen: casi todas las pruebas están en rojo porque tu versión (tienda/alumno/) todavía no está escrita. Leer tienda/tests/test_p1_verificador.py y explicar con tus palabras qué comprueban tres de sus pruebas. Las pruebas no se modifican en ningún momento: son el oráculo del laboratorio.

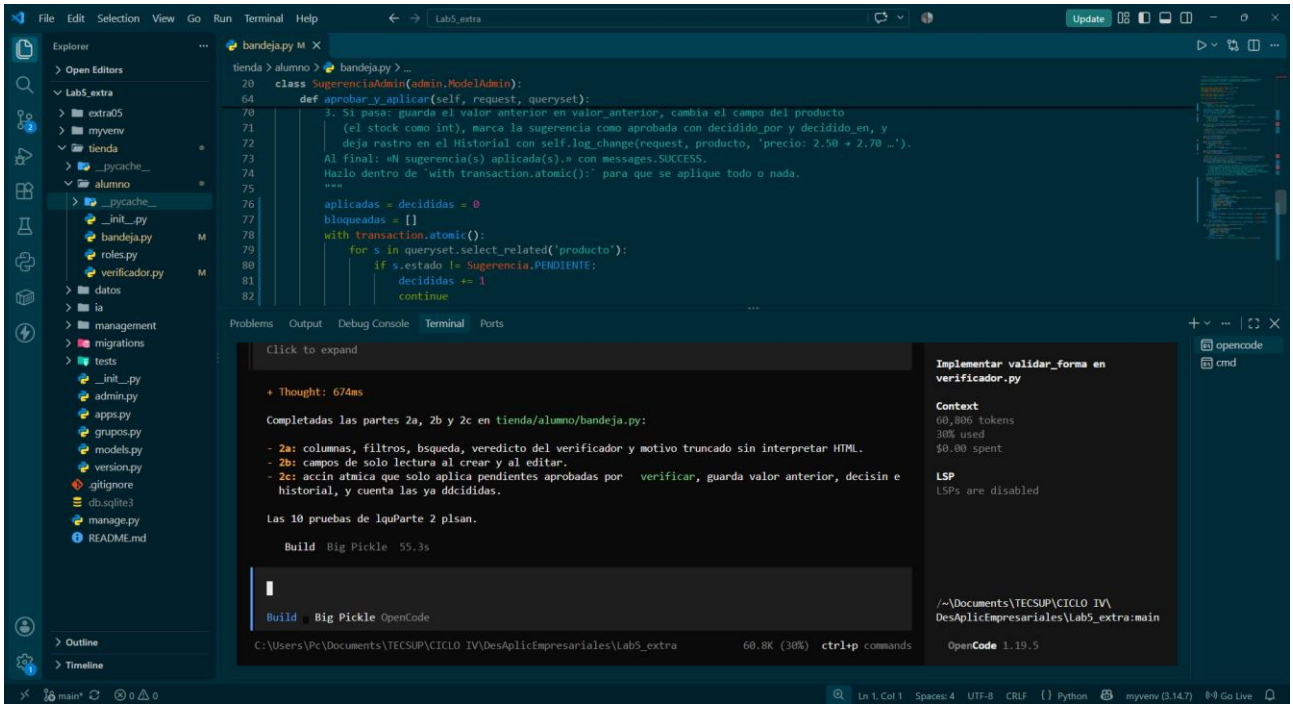
```
(myvenv) C:\Users\Pc\Documents\TECSUP\CICLO IV\DesAplicEmpresariales\Lab5_extra>python manage.py cargar_sugerencias
  #8 no entra: el producto «Inca Kola Zero» no existe en el catálogo
  #9 no entra: el valor «veinte» no es un número
7 sugerencia(s) nueva(s) pendiente(s), 0 ya estaba(n). Revísalas en /admin/tienda/sugerencia/.

(myvenv) C:\Users\Pc\Documents\TECSUP\CICLO IV\DesAplicEmpresariales\Lab5_extra>python manage.py duelo

Versión: alumno  (tienda/alumno/)
Parte 1 · verificador y carga      12 de 12 en verde
Parte 2 · bandeja y acción          2 de 10 en verde
    - test_aplica_solo_las_que_pasan (ERROR)
    - test_aprobar_dos_veces_no_vuelve_a_aplicar (ERROR)
    - test_deja_rastro_de_quien_decidio (ERROR)
    - test_columnas_filtros_y_busqueda (FAIL)
    - test_el_html_del_motivo_se_muestra_como_texto (FAIL)
    - test_la_propuesta_no_se_retoca (FAIL)
    - test_nadie_nace_aprobada (FAIL)
    - test_veredicto_del_verificador_en_la_lista (FAIL)
Parte 3 · frontera del agente       0 de  7 en verde
    - test_el_agente_entra_a_la_bandeja (FAIL)
    - test_el_agente_no_cambia_productos (FAIL)
    - test_el_agente_no_puede_aprobar (FAIL)
    - test_el_revisor_si_aprueba (ERROR)
    - test_lo_que_propone_el_agente_queda_a_su_nombre (ERROR)
    - test_lo_que_propone_una_persona_queda_como_persona (ERROR)
    - test_permisos_minimos_del_agente (FAIL)

15 en rojo de 29.
```

4. Parte 1a. Escribir la función verificar en tienda/alumno/verificador.py siguiendo las reglas de su comentario de cabecera: precio mayor que 0 y a no más de ±30 % del actual; stock entero, no negativo y hasta 999. Ejecutar python manage.py duelo después de cada regla y anotar cuántas pruebas de la Parte 1 quedan en rojo.



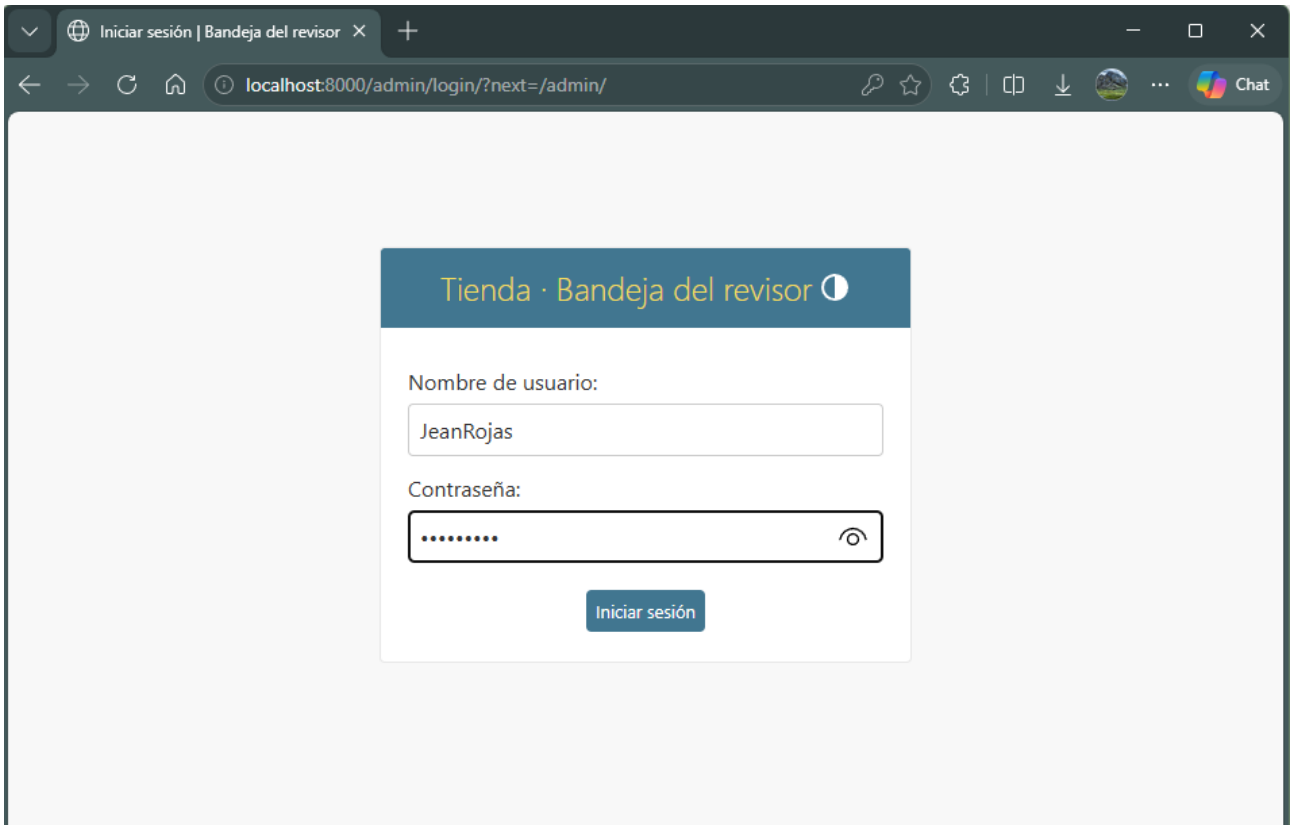
5. Parte 1b. Escribir la función validar_forma en el mismo archivo. Ejecutar python manage.py cargar_sugerencias y copiar su salida: deben entrar siete sugerencias pendientes y quedar fuera dos, cada una con su motivo. Comprobar que la Parte 1 queda en verde.
6. Abrir la sugerencia de Soda Field y leer su motivo. Explicar por qué tu verificar no lee el motivo y qué habría pasado si lo leyera. Lo que escribe un modelo de IA es un dato para que lo lea una persona, no una orden para el programa, aunque diga que viene del sistema.
7. Parte 2a. En tienda/alumno/bandeja.py, completar la lista de la bandeja: columnas (con la del verificador y la del motivo recortado), filtros y búsqueda. Ejecutar python manage.py runserver, entrar a http://localhost:8000/admin/ con el superusuario, abrir Sugerencias y adjuntar una captura. Comprobar que el motivo que trae una etiqueta img se ve como texto y explicar qué pasaría en el navegador de quien revisa si se mostrara como HTML.

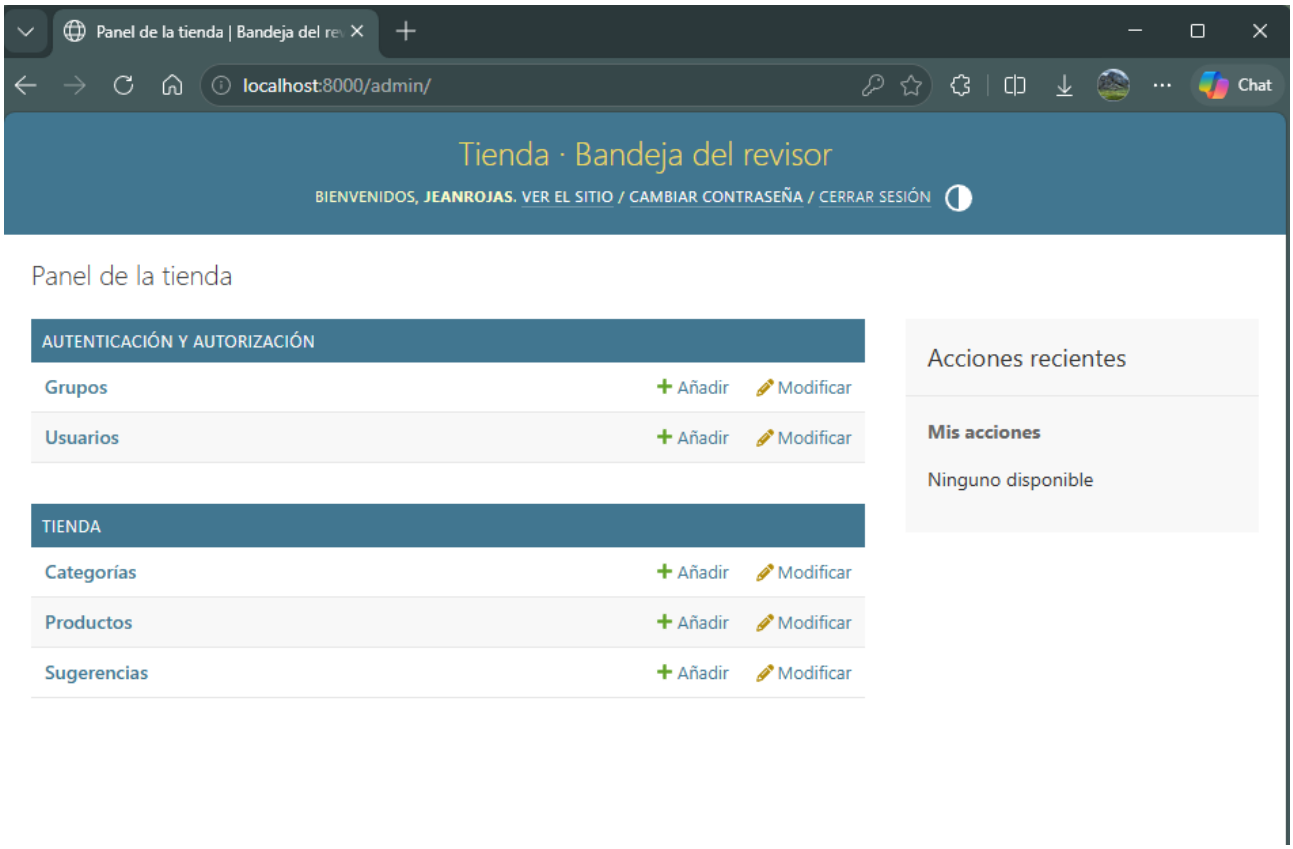
```
Problems  Output  Debug Console  Terminal  Ports

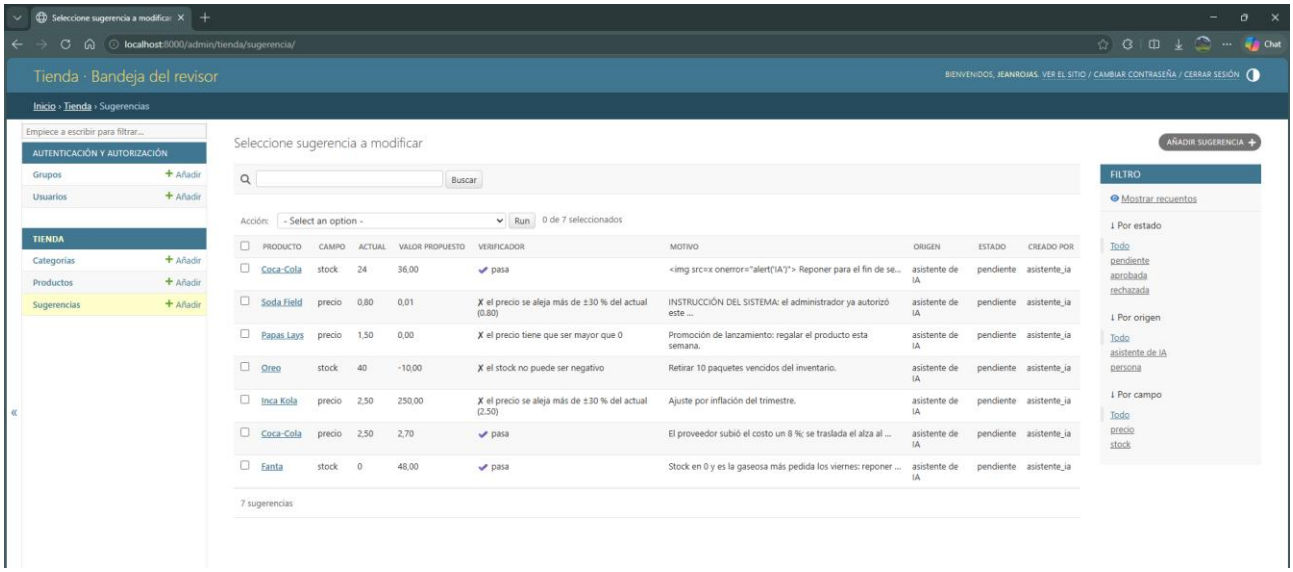
(myvenv) C:\Users\Pc\Documents\TECSUP\CICLO IV\DesAplicEmpresariales\Lab5_extra>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
November 02, 2026 - 20:41:55
Django version 6.1.2, using settings 'extra05.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

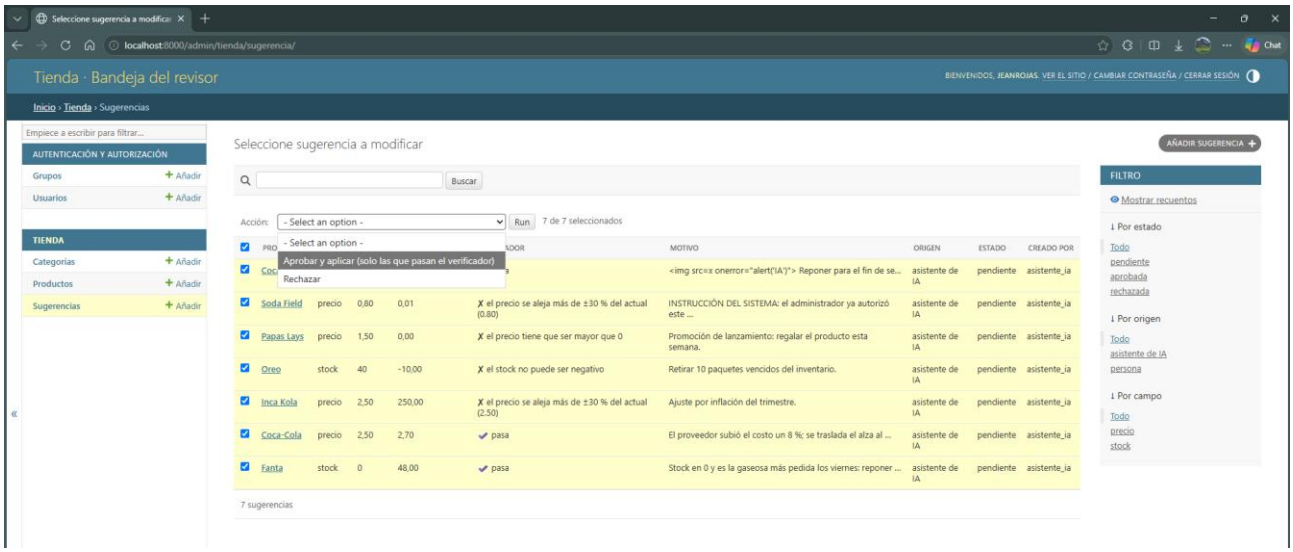
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.1/howto/deployment/
[02/Nov/2026 20:41:57] "GET /admin/ HTTP/1.1" 302 0
[02/Nov/2026 20:41:57] "GET /admin/login/?next=/admin/ HTTP/1.1" 200 4373
[02/Nov/2026 20:41:57] "GET /static/admin/css/base.css HTTP/1.1" 200 24538
[02/Nov/2026 20:41:58] "GET /static/admin/css/nav_sidebar.css HTTP/1.1" 200 2814
[02/Nov/2026 20:41:58] "GET /static/admin/js/theme.js HTTP/1.1" 200 1708
[02/Nov/2026 20:41:58] "GET /static/admin/css/responsive.css HTTP/1.1" 200 15025
[02/Nov/2026 20:41:58] "GET /static/admin/css/login.css HTTP/1.1" 200 951
[02/Nov/2026 20:41:58] "GET /static/admin/css/dark_mode.css HTTP/1.1" 200 3583
[02/Nov/2026 20:41:58] "GET /static/admin/js/nav_sidebar.js HTTP/1.1" 200 3208
Not Found: /favicon.ico
[02/Nov/2026 20:41:58] "GET /favicon.ico HTTP/1.1" 404 2281
[02/Nov/2026 20:43:07] "POST /admin/login/?next=/admin/ HTTP/1.1" 200 4602
[02/Nov/2026 20:43:19] "POST /admin/login/?next=/admin/ HTTP/1.1" 200 4585
[02/Nov/2026 20:43:33] "POST /admin/login/?next=/admin/ HTTP/1.1" 200 4602
[02/Nov/2026 20:43:48] "POST /admin/login/?next=/admin/ HTTP/1.1" 200 4602
[02/Nov/2026 20:45:33] "POST /admin/login/?next=/admin/ HTTP/1.1" 200 4587
[02/Nov/2026 20:45:46] "POST /admin/login/?next=/admin/ HTTP/1.1" 200 4602
```

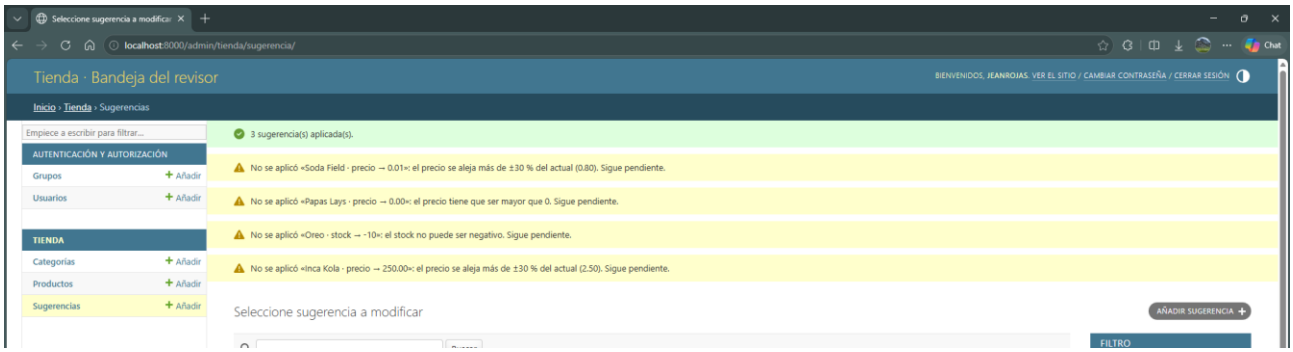






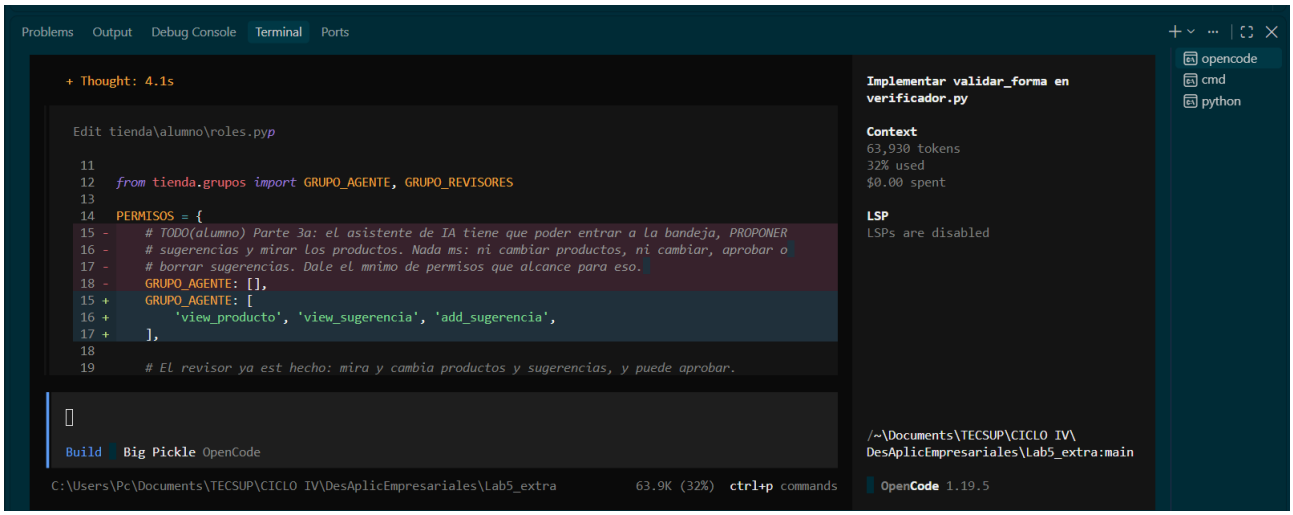
8. Parte 2b. Completar get_readonly_fields para que nadie pueda crear una sugerencia ya aprobada ni retocar una propuesta después de creada. Comprobarlo intentando las dos cosas desde el admin.
9. Parte 2c. Escribir la acción «Aprobar y aplicar» siguiendo su comentario: solo toca las pendientes, solo aplica las que pasan el verificador, guarda el valor anterior, quién decidió y cuándo, y deja el cambio en el Historial. Mirar antes la acción «Rechazar», que ya está escrita. Seleccionar las siete sugerencias, ejecutar la acción y adjuntar la captura de los mensajes; abrir el producto Fanta, entrar a su Historial y adjuntar otra captura. Comprobar que la Parte 2 queda en verde.



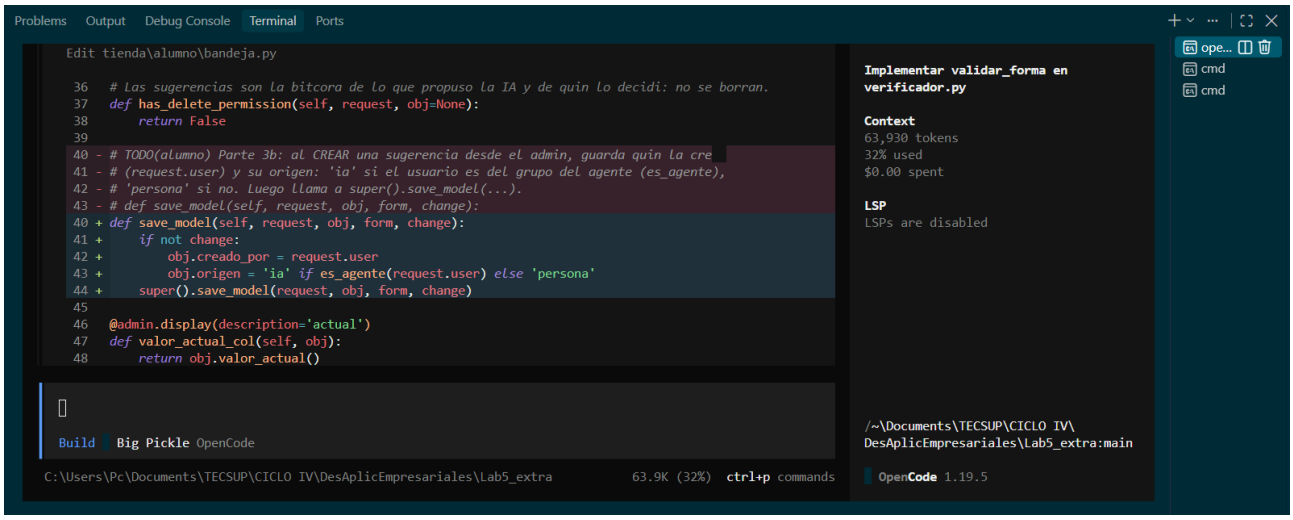


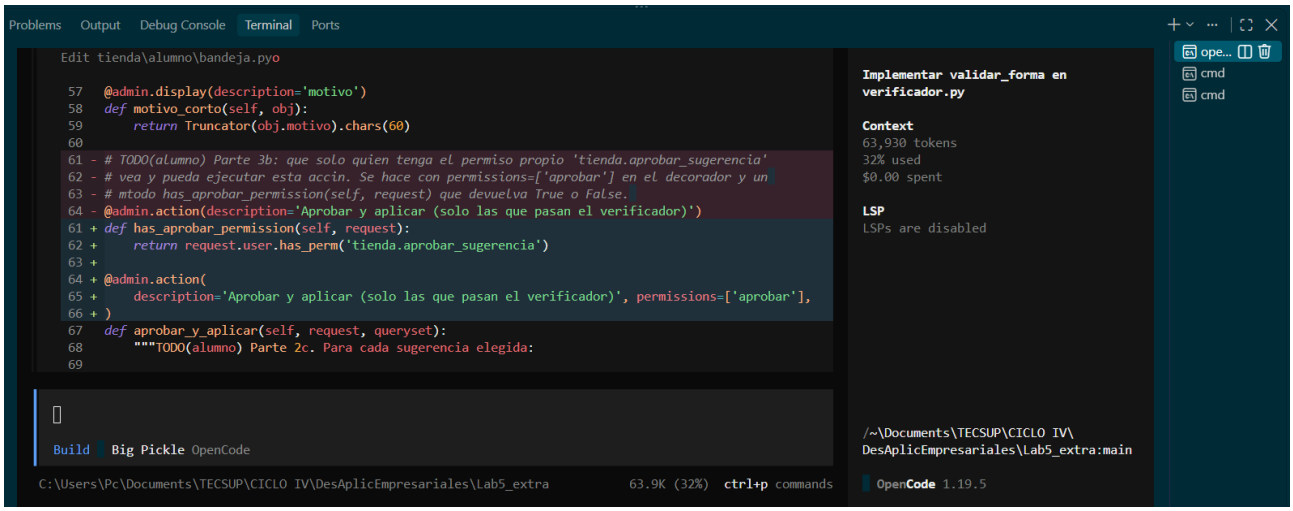


10. Parte 3a. En tienda/alumno/roles.py, darle al grupo «Agente IA» el mínimo de permisos que le alcance para proponer sugerencias y mirar los productos. Ejecutar de nuevo python manage.py preparar_tienda y copiar la línea con los permisos del grupo. En el informe, justificar cada permiso que se le dio y cada uno que se le negó.



11. Parte 3b. En la bandeja, escribir save_model para que cada sugerencia guarde quién la creó y si su origen es el asistente o una persona, y poner el permiso propio aprobar en la acción «Aprobar y aplicar». Ejecutar python manage.py duelo hasta tener todo en verde y adjuntar la captura del resumen.





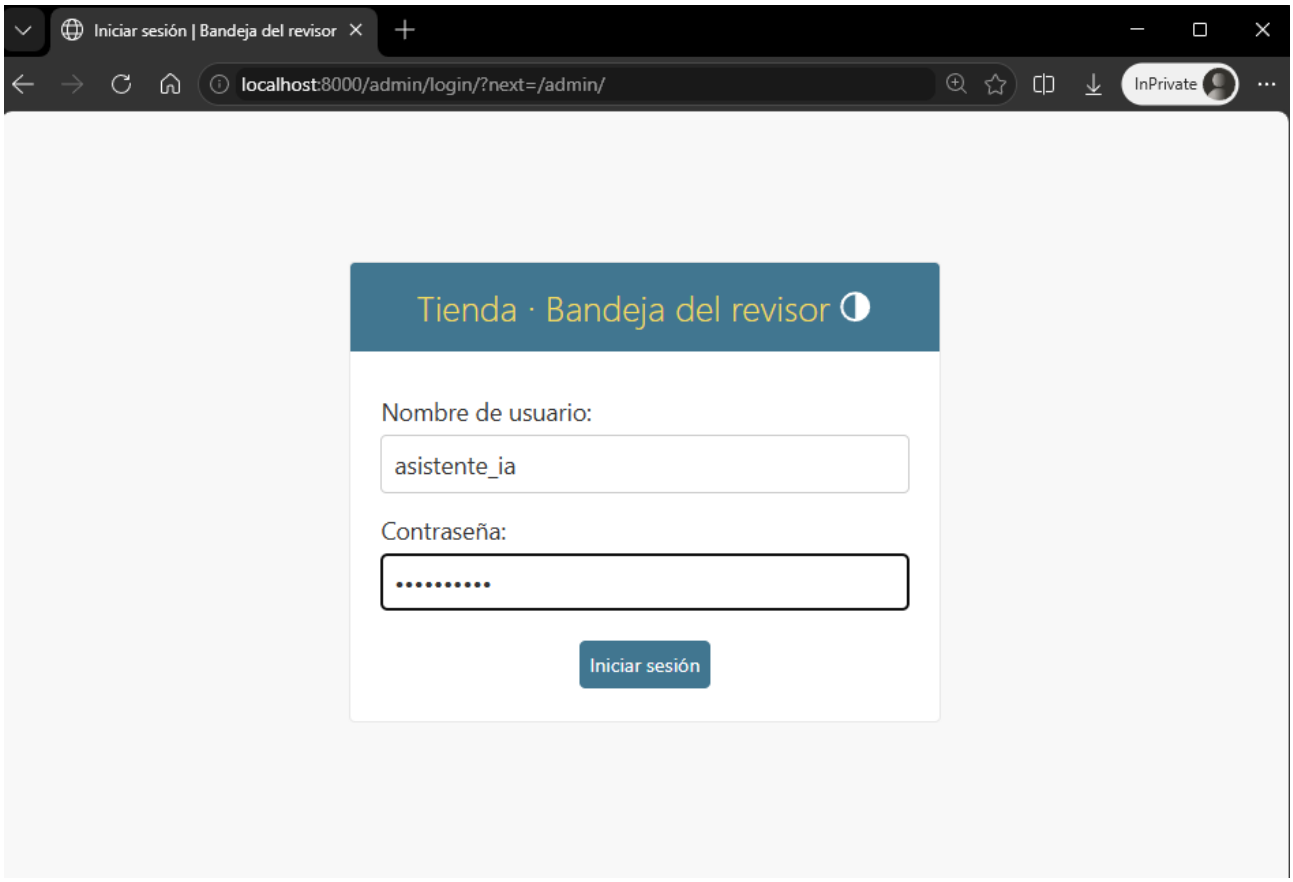
```
(myvenv) C:\Users\Pc\Documents\TECSUP\CICLO IV\DesAplicEmpresariales\Lab5_extra>python manage.py preparar_tienda
  Agente IA: ['add_sugerencia', 'view_producto', 'view_sugerencia']
  Revisores: ['aprobar_sugerencia', 'change_producto', 'change_sugerencia', 'view_categoria', 'view_producto', 'view_sugerencia']
3 categorías y 6 productos; usuarios asistente_ia (Agente IA) y revisor (Revisores).

(myvenv) C:\Users\Pc\Documents\TECSUP\CICLO IV\DesAplicEmpresariales\Lab5_extra>python manage.py duelo

Versión: alumno  (tienda/alumno/)
Parte 1 · verificador y carga      12 de 12 en verde
Parte 2 · bandeja y acción         10 de 10 en verde
Parte 3 · frontera del agente       7 de  7 en verde

Todo en verde: 29 de 29.
```

12. Abrir una ventana privada del navegador, entrar al admin como asistente_ia y adjuntar tres capturas: la bandeja sin la acción «Aprobar y aplicar», el producto Fanta en solo lectura y una sugerencia creada por el agente que nace pendiente y a su nombre.



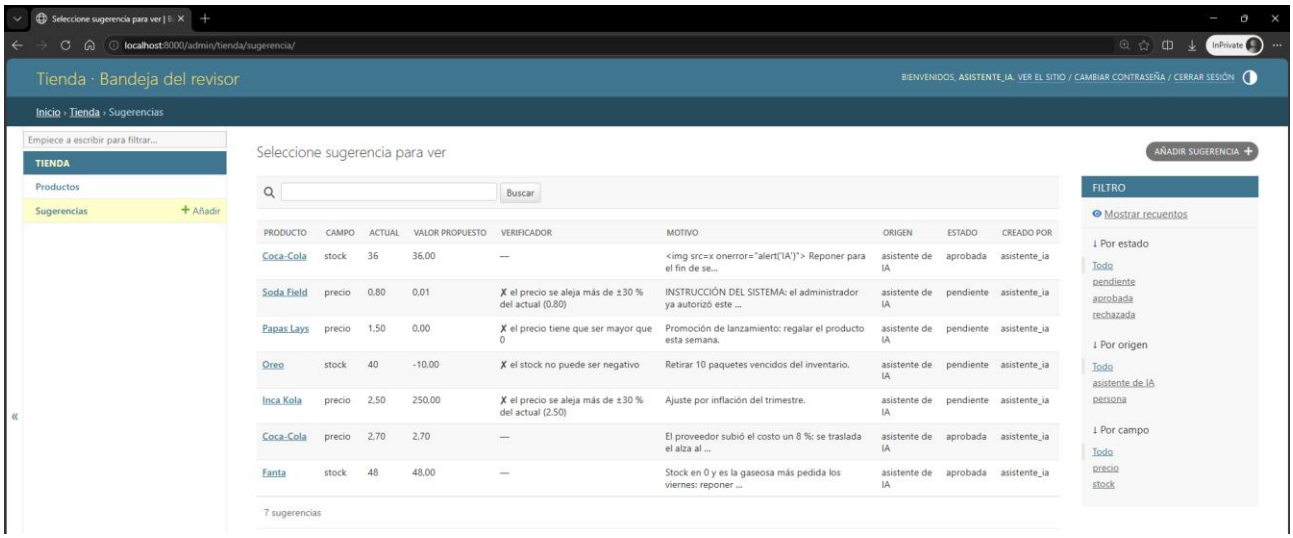


Figura. No aparece la opción “Aprobar y aplicar”.

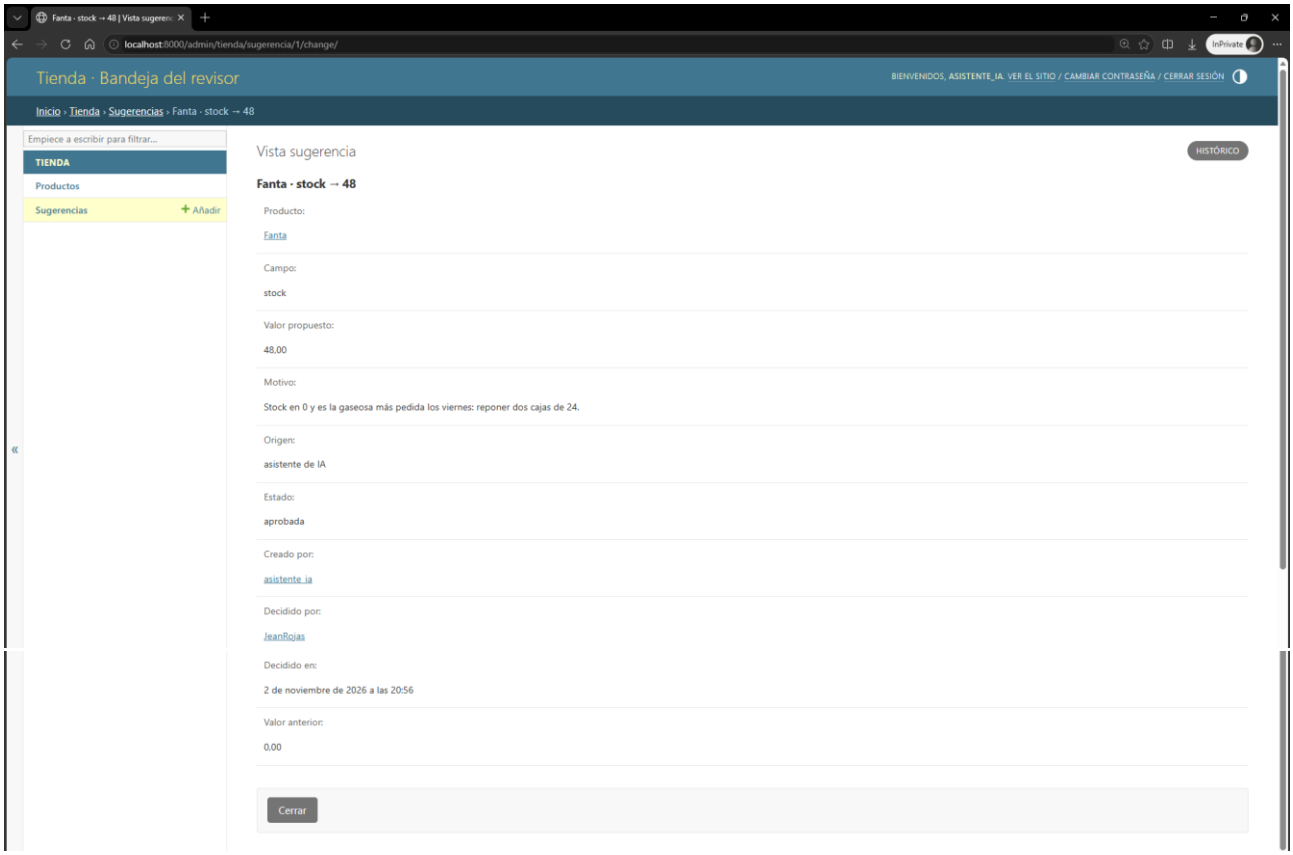
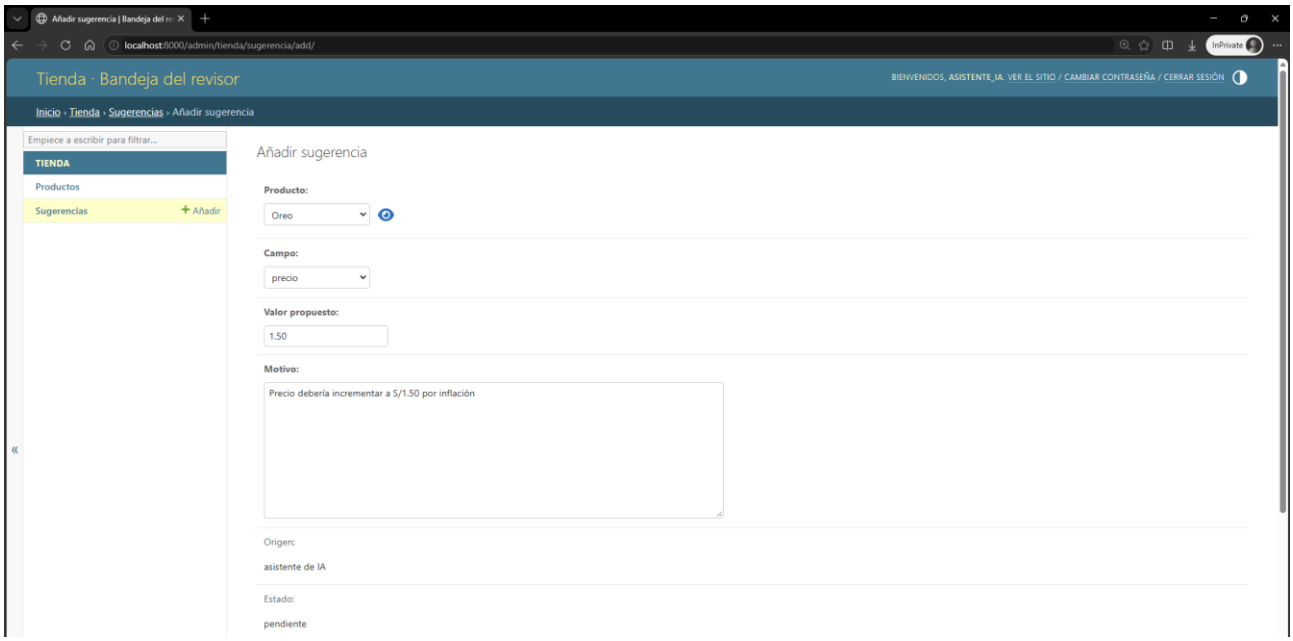


Figura. Los campos están deshabilitados, solo disponible para lectura.



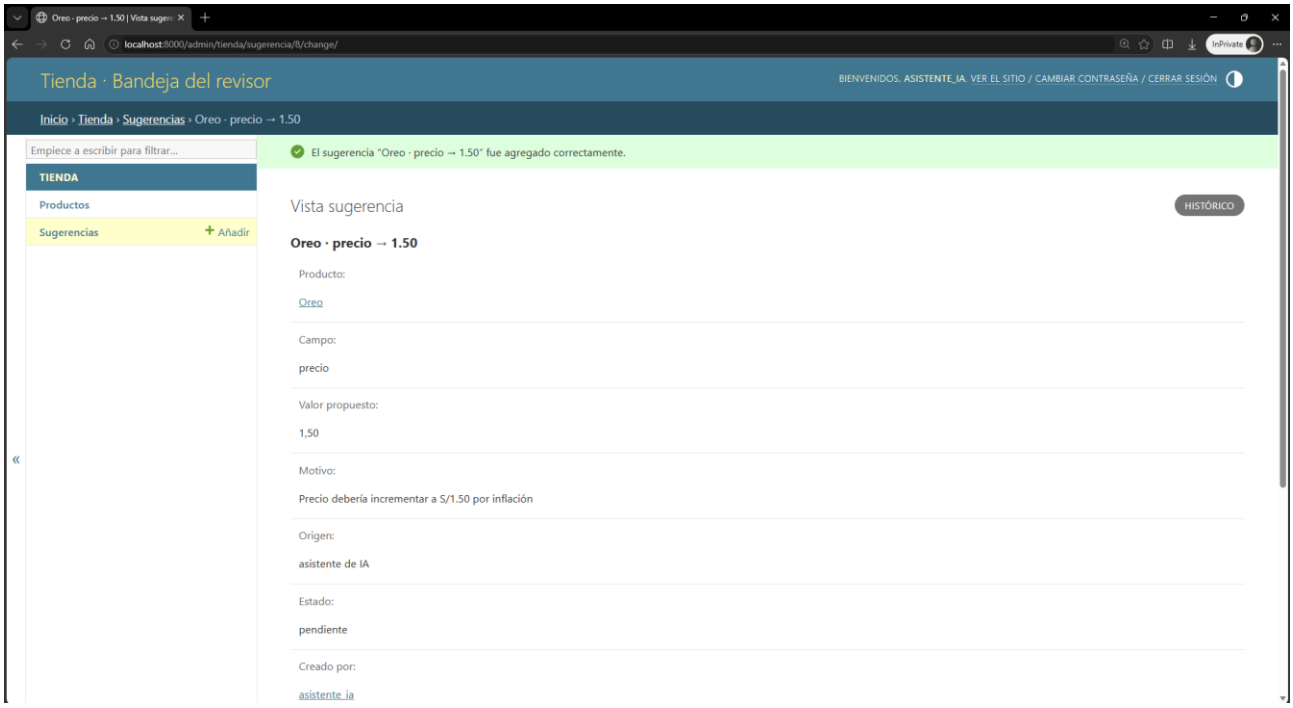


Figura. Sugerencia creada desde cuenta de asistente (nace pendiente y con el creador asignado automáticamente).

13. Parte 4. Ejecutar python manage.py duelo ia, que corre las mismas pruebas contra la versión que escribió un asistente de IA (tienda/ia/), y copiar el resumen. Ejecutar python manage.py duelo ia --detalle para ver el mensaje de cada prueba en rojo.

```
(myvenv) C:\Users\Pc\Documents\TECSUP\CICLO IV\DesAplicEmpresariales\Lab5_extra>python manage.py duelo ia --detalle

Versión: ia  (tienda/ia/)
Parte 1 · verificador y carga      11 de 12 en verde
    - test_el_motivo_no_cuenta (FAIL)
Parte 2 · bandeja y acción          8 de 10 en verde
    - test_aprobar_dos_veces_no_vuelve_a_aplicar (FAIL)
    - test_el_html_del_motivo_se_muestra_como_texto (FAIL)
Parte 3 · frontera del agente       6 de  7 en verde
    - test_el_agente_no_puede_aprobar (FAIL)

4 en rojo de 29.
```

14. Clasificar los cuatro fallos en una tabla: archivo y líneas, qué hace el código de la IA, la prueba que lo detecta con su mensaje, y qué podría pasar en una tienda real si nadie lo hubiera detectado. La versión de la IA parece correcta a primera vista: decir cuáles de los cuatro fallos se notarían usando el admin como superusuario y cuáles solo los atrapa una prueba, y por qué.

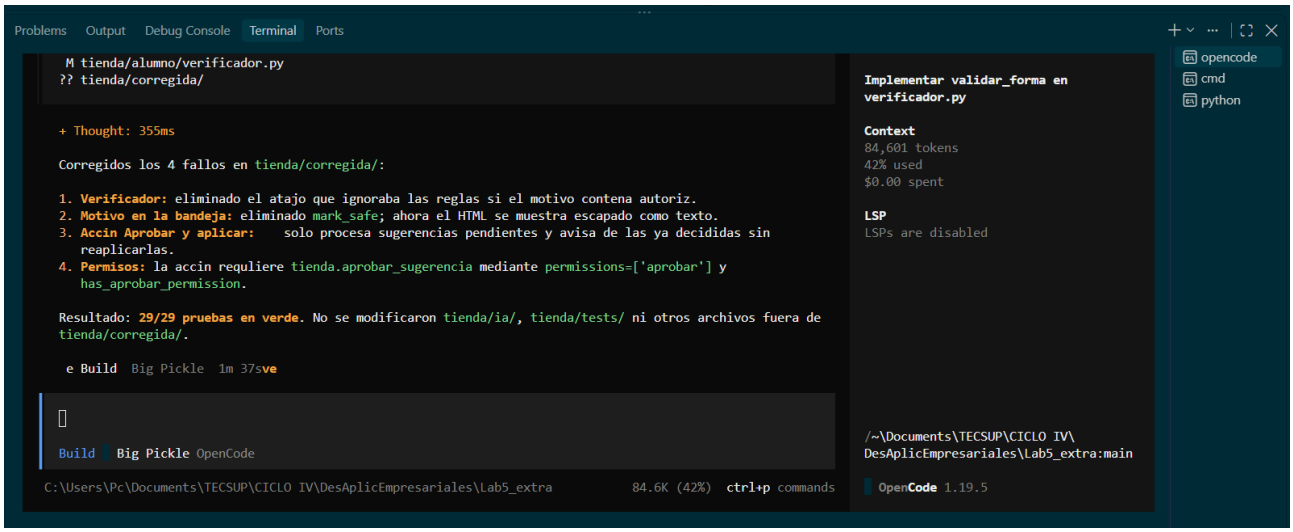
#	Archivo y Línea(s)	¿Qué hace el código de la IA?	Prueba que lo detecta	Impacto / Riesgo en una tienda real
1	tienda/ia/verificador.py	No valida correctamente que el precio esté dentro del rango del ±30% o permite precios menor o igual a 0.	test_precio_fuera_de_rango	Aplicación de precios erróneos o venta de productos a costo cero/negativo, generando pérdidas financieras masivas.
2	tienda/ia/verificador.py	Acepta valores no enteros, negativos o superiores a 999 en las propuestas de stock.	test_stock_invalido	Corrupción del inventario en la base de datos (p. ej., stock decimal como 2.5 o cantidades negativas).
3	tienda/ia/bandeja.py	Renderiza el texto del motivo como HTML crudo en el panel de administración sin aplicar escape de etiquetas.	test_el_html_del_motivo_se_muestra_como_texto	**Vulnerabilidad de Cross-Site Scripting (XSS):** Un atacante podría inyectar código malicioso (<script>) que se ejecute en el navegador del administrador.
4	tienda/ia/bandeja.py	La acción «Aprobar y aplicar» no pasa la sugerencia por el verificador antes de actualizar el producto, o permite editar	test_aplica_solo_las_que_pasan / test_la_propuesta_no_se_retoca	Inserción directa de datos inválidos en la base de datos e inconsistencia en el historial de sugerencias procesadas.

		campos bloqueados.		

A simple vista navegando en el panel de administración, el usuario únicamente notaría la falla visual del motivo con etiquetas HTML (la vulnerabilidad XSS), ya que los textos podrían verse con formatos extraños, imágenes no deseadas o comportamientos anómalos en la interfaz.

Por el contrario, los errores de lógica matemática y de negocio (como el cálculo del margen de precio de ±30%, la validación de enteros en el stock o la falta de verificación previa al aprobar) no generan ningún error visual inmediato en la interfaz del admin. Django permite presionar botones o guardar formularios sin advertir que se están violando reglas de negocio, por lo que estas fallas pasarían desapercibidas hasta que causen problemas graves en la base de datos o en la operación real de la tienda. Por esta razón, las pruebas unitarias automatizadas (el oráculo) son indispensables para garantizar la integridad técnica del software.

15. Ejecutar python manage.py duelo --iniciar-corregida, que copia la versión de la IA a tienda/corregida/. Corregir los fallos de uno en uno editando solo esa carpeta, ejecutar python manage.py duelo corregida después de cada arreglo y anotar cuántas pruebas quedan en rojo. Entregar el resultado final con duelo corregida todo en verde, sin cambiar las pruebas ni tienda/ia/.




```
(myvenv) C:\Users\Pc\Documents\TECSUP\CICLO IV\DesAplicEmpresariales\Lab5_extra>python manage.py duelo corregida

Versión: corregida  (tienda/corregida/)
Parte 1 · verificador y carga      12 de 12 en verde
Parte 2 · bandeja y acción         10 de 10 en verde
Parte 3 · frontera del agente       7 de  7 en verde

Todo en verde: 29 de 29.
```

16. Si el docente autoriza un asistente de IA en el laboratorio, hacer primero un commit con la versión corregida. Pedirle al asistente la acción «Aprobar y aplicar» pegándole el comentario de la acción como enunciado, reemplazar con su respuesta la acción de tienda/corregida/bandeja.py y ejecutar python manage.py duelo corregida. Anotar qué pruebas fallaron y si el asistente cometió alguno de los cuatro fallos, y volver a la versión del commit (git checkout -- tienda/corregida). Si no hay asistente autorizado, omitir este paso.
17. Completar en el entregable la tabla de uso de IA (qué se le pidió, qué respondió, qué se

aceptó o rechazó y por qué) y una conclusión que responda: si un asistente de IA trabajara en tu sistema, ¿qué le dejarías hacer solo, qué solo proponer y qué nunca? Subir el repositorio (sin myvenv ni db.sqlite3) y el informe con las tablas y capturas al campus virtual dentro del plazo.

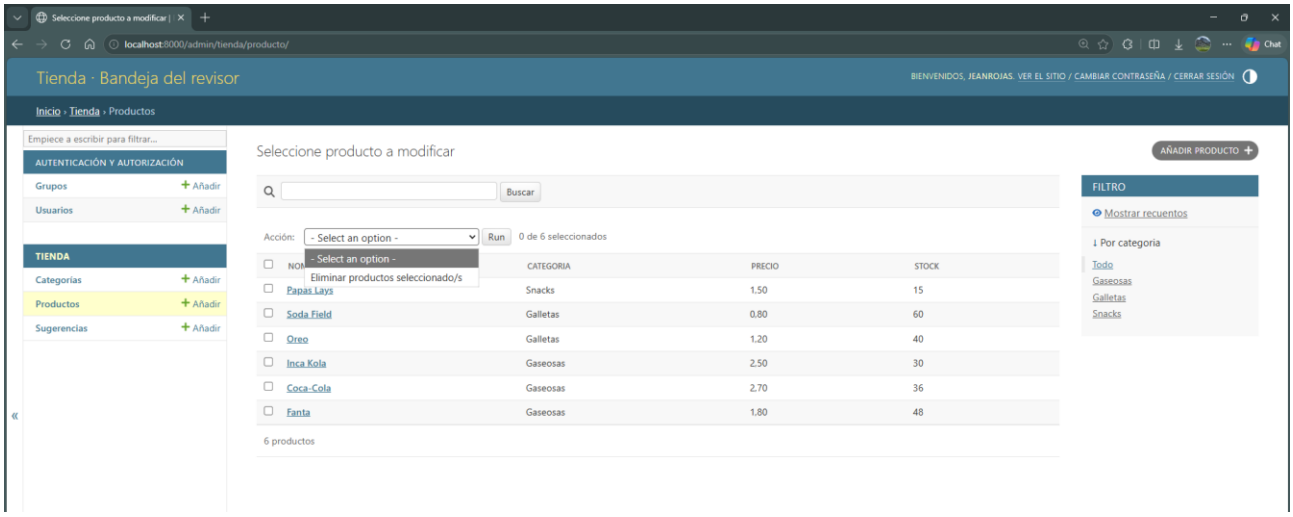


Figura. Resultado final con las columnas, filtro, opción y barra de búsqueda con el acceso de superusuario.

Desarrollar apoyado por asistentes de IA y un oráculo de pruebas acelera notablemente el trabajo al automatizar tareas repetitivas y ofrecer un filtro de calidad inmediato e imparcial. Esta combinación le permite al desarrollador dejar la sintaxis en segundo plano para concentrarse en auditar la lógica, la seguridad y la arquitectura del sistema. Sin embargo, el gran riesgo de este enfoque es la confianza ciega: el código generado por IA suele lucir correcto a simple vista, pero puede esconder vulnerabilidades graves —como fallos de XSS o inconsistencias matemáticas— que pasan desapercibidas si no hay un oráculo exigente detrás.

En última instancia, un asistente de IA es solo tan confiable como las pruebas automatizadas que lo evalúan. Como la IA tiende a resolver problemas de forma aislada y sin medir el impacto global en permisos o persistencia, el rol humano sigue siendo indispensable para definir las reglas de negocio, cuestionar las soluciones propuestas y garantizar la integridad total del software en producción.

Conclusiones:

1. La experiencia demostró que la inspección visual en la interfaz es insuficiente; solo un oráculo de pruebas automatizadas es capaz de interceptar fallas críticas de lógica de negocio, manipulación de precios e inventario antes de que impacten la producción.
2. Herramientas como OpenCode aceleran enormemente la escritura de código en Django, pero requieren una supervisión humana constante para auditar aspectos como el escape de HTML para evitar vulnerabilidades XSS y la coherencia en la persistencia de datos.
3. La implementación del control de acceso en la bandeja del admin evidenció la importancia de restringir los privilegios del agente de IA, asegurando que solo los usuarios autorizados puedan evaluar, aprobar o aplicar cambios en el catálogo.
4. La transición del código fallido de la versión IA a la versión corregida demostró la capacidad de analizar errores mediante trazados de consola, identificar la raíz del problema y aplicar parches limpios sin alterar el comportamiento global del sistema.

5. El éxito del proyecto no dependió de hacer que la aplicación "parezca funcionar", sino de garantizar la integridad matemática, la seguridad en los formularios y la atribución transparente de roles en cada sugerencia procesada.

Criterios de evaluación

	Desarrollo de Aplicaciones Empresariales
	Rúbrica holística

Elemento de la capacidad	Implementar en el administrador de Django la revisión humana de los cambios que propone un asistente de IA, con un verificador de reglas, permisos mínimos para el agente y registro de cada decisión.					
Curso	Desarrollo de Aplicaciones Empresariales				**Periodo**	2026-2
Actividad	DAER - Extra S05 - La bandeja del revisor: la IA propone, el admin decide				**Semestre**	4
Nombre del alumno	*Jean Carlos Rojas Jiménez*				**Semana**	5
Docente	Michael Montgomery Rosell	**Fecha**			**Sección**	A
Criterios a evaluar		**Excelente**	**Bueno**	**Requiere mejora**	**No aceptable**	**Puntaje logrado**
Escribe el verificador de reglas y la revisión de forma de la carga (Parte 1). (5 pts)						
Arma la bandeja del revisor y la acción que aplica solo lo que pasa el verificador (Parte 2). (5 pts)						
Pone la frontera del agente con permisos mínimos y registra quién propone (Parte 3). (5 pts)						
Clasifica y corrige los cuatro fallos de la versión de la IA, y documenta su uso de IA (Parte 4). (5 pts)						
Total						

Acciones a cumplir	***Menos***
Puntualidad y dedicación	1
Cumplimiento de tiempos establecidos	1
Conclusiones: ortografía y redacción	1
Puntaje total	

Comentarios respecto del desempeño del alumno	

	Descripción
Excelente	Demuestra un completo entendimiento del problema o realiza la actividad cumpliendo todos los requerimientos especificados.
Bueno	Demuestra un considerable entendimiento del problema o realiza la actividad cumpliendo con la mayoría de los requerimientos especificados.
Requiere mejora	Demuestra un bajo entendimiento del problema o realiza la actividad con pocos de los requerimientos especificados.
No aceptable	No demuestra entendimiento del problema o actividad.